

API and Backend Development

Contents

API fundamentals

1. [What an API is](#)
2. [HTTP essentials](#)
3. [Status codes](#)
4. [REST](#)

Designing the API

5. [Resource modelling and URL design](#)
6. [Safety and idempotency](#)
7. [Pagination, filtering and sorting](#)
8. [Validation and error responses](#)
9. [Versioning](#)

Beyond REST

10. [REST, gRPC and GraphQL](#)

Security

11. [Authentication and authorization](#)
12. [API security](#)
13. [Rate limiting](#)

Reliability

14. [Timeouts and retries](#)
15. [Idempotency keys](#)
16. [Caching](#)

Building it in Java

17. [Layering a Spring Boot service](#)
18. [Error handling](#)
19. [Testing an API](#)
20. [Observability](#)

Reference

- 21. [Interview questions](#)
- 22. [Common mistakes](#)
- 23. [Quick reference](#)

1. What an API is

An **API, Application Programming Interface**, is a contract between two pieces of software. It states what a caller may ask for, what shape the answer takes, and what can go wrong. Backend development is building the side that honours that contract.

The framing worth carrying into an interview: **the contract is the product**. Internally we can rewrite whatever we like, and none of it is visible. The moment a field name, a status code or an error shape is published, callers depend on it, and most of those callers are not ours to update. That single asymmetry explains versioning, backward compatibility, and why field names get argued over.

2. HTTP essentials

HTTP, HyperText Transfer Protocol, is a request and response protocol. A request carries a method, a path, headers and usually a body. A response carries a status code, headers and usually a body.

Method	Purpose	Safe	Idempotent
GET	Read a resource	Yes	Yes
HEAD	Like GET , headers only	Yes	Yes
OPTIONS	Ask what is allowed	Yes	Yes
POST	Create, or anything that does not fit	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Change part of a resource	No	Not necessarily
DELETE	Remove a resource	No	Yes

Headers that appear repeatedly:

Header	Use
Content-Type	What the body is, such as <code>application/json</code>
Accept	What the client will take back
Authorization	Credentials, usually <code>Bearer <token></code>
Cache-Control	How and how long a response may be cached

Header	Use
<code>ETag</code> and <code>If-None-Match</code>	Version tag, and conditional request against it
<code>Location</code>	Where a newly created resource lives
<code>Retry-After</code>	How long to wait after a 429 or 503

HTTP/1.1 does define pipelining, but browsers do not rely on it, so in practice one request is in flight per connection and browsers open several. HTTP/2 multiplexes many streams over one connection and compresses headers, which removes most of that overhead. It matters here for one practical reason: gRPC requires HTTP/2, which is part of why browsers cannot call it directly.

3. Status codes

The first digit gives the class: **2xx** succeeded, **3xx** more action is needed, **4xx** the request cannot be fulfilled as submitted, and **5xx** the server failed to fulfil a request that was otherwise valid.

Code	Meaning	Typical use
200 OK	Succeeded	A read, or an update returning the result
201 Created	A resource now exists	Creation, with a <code>Location</code> header pointing at it
202 Accepted	Taken, not finished	Work queued for later processing
204 No Content	Succeeded, nothing to return	A delete, or an update returning nothing
400 Bad Request	Malformed	Unparseable body, missing required parameter
401 Unauthorized	Not authenticated	No credentials, or invalid ones
403 Forbidden	Authenticated, not allowed	Valid identity, insufficient permission
404 Not Found	No such resource	Also used to hide existence from an unauthorized caller
409 Conflict	Conflicts with current state	Duplicate creation, version conflict
422 Unprocessable Content	Understood, semantically wrong	Valid JSON that breaks a business rule
429 Too Many Requests	Rate limited	Send <code>Retry-After</code> with it
500 Internal Server Error	An unexpected server-side failure	Retrying may or may not help, depending on the cause
502 Bad Gateway	A gateway received an invalid response from upstream	A dependency answered with something unusable
503 Service Unavailable	Temporarily unable	Overloaded, or shutting down

Code	Meaning	Typical use
504 Gateway Timeout	An upstream did not answer in time	A dependency was too slow

Three distinctions that get asked directly.

401 against 403. 401 says the caller has not been identified, so either no credentials arrived or they were not valid. 403 says the caller has been identified and is refused anyway. The name of 401 is historically confusing, since it reads Unauthorized but means unauthenticated.

400 against 422. 400 is for a request the server cannot parse or that is missing something structural. 422 is for a request that parsed fine and breaks a rule, such as an order total of zero. 422 comes from the WebDAV extension rather than base HTTP, so it is widely used but not universal, and returning 400 for both is a defensible choice as long as it is consistent.

Never return 200 with an error inside the body. Every caller, proxy, monitor and retry policy reads the status code, and a success code on a failure defeats all of them.

4. REST

REST, REpresentational State Transfer, is an architectural style described by Roy Fielding in 2000. It is a set of constraints rather than a specification:

- **Client and server are separate**, and evolve independently.
- **Stateless.** Each request carries everything needed to process it, so the server does not depend on client session state held between requests. That is about session state, not about storing nothing at all; the database is still there.
- **Cacheable.** Responses say whether they may be cached.
- **Uniform interface.** Resources are identified by URL, manipulated through representations, and responses are self-descriptive.
- **Layered system.** A client cannot tell whether it is talking to the server or to a proxy in front of it.
- **Code on demand**, optional and rarely used.

Statelessness is the constraint with the practical consequence: because no request depends on a session held in one server's memory, any instance can serve any request, which is what makes horizontal scaling and rolling deploys straightforward.

It also does not mean the API is unauthenticated, which is a question worth being ready for. The credentials travel with each request instead of being remembered between them, as a bearer token or a session identifier the server can validate on its own.

Worth being honest about, because it is a good interview answer: **most APIs called REST are HTTP JSON APIs.** The uniform interface constraint includes hypermedia, where responses carry links telling the client what it can do next, and almost nobody implements that. The **Richardson Maturity Model** describes the ladder: level 0 is

one endpoint doing everything, level 1 introduces resources, level 2 uses HTTP methods and status codes properly, and level 3 adds hypermedia. Most production APIs sit at level 2 and stop there deliberately, because the cost of level 3 rarely pays back.

5. Resource modelling and URL design

A resource is a noun. The method supplies the verb.

GET	/orders	list orders
POST	/orders	create one
GET	/orders/{id}	read one
PUT	/orders/{id}	replace it
PATCH	/orders/{id}	change part of it
DELETE	/orders/{id}	remove it
GET	/customers/{id}/orders	orders belonging to one customer

Conventions that hold up in review:

- **Plural collection names.** `/orders`, not `/order`.
- **No verbs in the path for ordinary operations.** `POST /orders` rather than `/createOrder`, because the method already says create.
- **Nest one level at most.** `/customers/{id}/orders` is useful.
`/customers/{id}/orders/{id}/items/{id}` is a URL nobody can maintain, and the deeper resources have their own top-level paths anyway.
- **Consistent naming for identifiers, dates and casing** across every endpoint, since inconsistency is what makes an API tiring to use.

Some operations are genuinely not create, read, update or delete. Cancelling an order is a state transition with its own rules, and modelling it as `PATCH /orders/{id}` with `{"status": "cancelled"}` invites callers to set any status at all. The pragmatic answer, widely used and not strictly REST, is a sub-resource that names the action:

```
POST /orders/{id}/cancel
```

Being able to say "this is a deliberate departure, and here is why" is a better answer than pretending everything fits CRUD.

6. Safety and idempotency

Two properties from the HTTP specification, constantly confused, and the reason retries can be safe at all.

Safe means the request does not change server state. `GET`, `HEAD` and `OPTIONS` are safe. A `GET` that changes something is a real bug, because crawlers, prefetchers and browsers assume otherwise.

Idempotent means making the same request several times leaves the server in the same state as making it once. `GET`, `PUT`, `DELETE`, `HEAD` and `OPTIONS` are idempotent. `POST` is not. `PATCH` is not necessarily, and whether it is depends on how it is written: setting a field to a value is idempotent, incrementing it is not.

The nuance an interviewer may push on: `DELETE` is idempotent even though the first call returns 204 and the second returns 404. Idempotency is about the resulting **state**, not about the response being identical.

Why it matters: a client whose request times out does not know whether the server processed it. Retrying an idempotent request is safe. Retrying a `POST` may create a second order or take a second payment, which is what section 15 exists to solve.

7. Pagination, filtering and sorting

Design any collection endpoint that can grow for pagination from the start. Adding it later becomes a breaking change as soon as it alters the response shape or the default behaviour, and returning a bare JSON array is what makes that likely, because wrapping it in an object with the page metadata changes the type every client parses.

Offset pagination is the familiar form:

```
GET /orders?page=2&size=20
```

It is simple and allows jumping to any page. Two costs: the database must walk and discard every skipped row, so deep pages get slower, and if rows are inserted between requests the pages shift, so an item can appear twice or be missed.

Keyset pagination, also called cursor pagination, uses the last item seen as the starting point:

```
GET /orders?after=2026-08-15T10:00:00Z_1042&limit=20
```

The cost is flat regardless of depth, and results stay stable while data changes underneath. The trade is that jumping straight to page fifty is no longer possible, which is usually acceptable for feeds and rarely acceptable for an admin table.

The cursor above is deliberately compound. Keyset pagination needs a stable, deterministic ordering, which usually means a unique tie-breaker alongside the sort column, so `(createdAt, id)` rather than `createdAt` alone. Two rows sharing a timestamp would otherwise have no defined order between them, and the page boundary could skip or repeat one.

Whichever is chosen, **cap the page size on the server**. A caller asking for a million rows should get the maximum, not a timeout.

For filtering and sorting, keep it explicit and allow-listed:

```
GET /orders?status=shipped&createdAfter=2026-01-01&sort=createdAt,desc
```

Passing a sort field straight into a query is how an ordering parameter becomes an injection point, so map incoming names onto known columns rather than trusting them.

8. Validation and error responses

Validation happens at more than one layer, and each catches a different class of problem. The **shape** of a request is checked at the edge, because that is the trust boundary and it is where a malformed body should become a clear error rather than an exception. **Business invariants** belong in the service or domain layer, where the rules live. The invariants that must hold whatever writes the data belong in **database constraints**, because the application is not the only writer.

```
public record CreateOrderRequest(  
    @NotNull Long customerId,  
    @NotNull @Positive BigDecimal total,  
    @Email String notifyEmail) { }
```

```
@PostMapping("/orders")  
@ResponseStatus(HttpStatus.CREATED)  
public OrderResponse create(@Valid @RequestBody CreateOrderRequest request) {  
    return OrderResponse.from(orders.create(request.toCommand()));  
}
```

`@Valid` triggers Bean Validation, and a failure becomes a `MethodArgumentNotValidException` that section 18 turns into a response.

Errors need a consistent shape across every endpoint. **RFC 9457, Problem Details for HTTP APIs**, which replaced RFC 7807, defines one, and Spring supports it directly through `ProblemDetail`:

```
{  
    "type": "https://api.example.com/problems/insufficient-funds",  
    "title": "Insufficient funds",  
    "status": 409,  
    "detail": "Account 42 has 15.00 available, and 20.00 was requested",  
    "instance": "/accounts/42/withdrawals",  
    "traceId": "0af7651916cd43dd8448eb211c80319c"  
}
```

Three rules for error bodies:

- **A machine-readable code**, in `type` or a dedicated field, so callers branch on something stable. Nothing should ever branch on `detail`, which is prose written for a human and free to be reworded.
- **A correlation identifier**, so a user reporting a failure can be matched to the logs for that request.
- **No internal detail**. Stack traces, SQL fragments and class names help an attacker map the system and help the caller not at all.

9. Versioning

The cheapest version is the one never needed, so start by separating changes that break callers from changes that do not.

Not breaking, for tolerant clients: adding a new endpoint, adding an optional request field, adding a response field. That holds only because clients ignore what they do not recognise, and a strict client that rejects unknown fields still breaks, so tolerant readers are worth insisting on early and documenting as an expectation.

Breaking: removing or renaming a field, changing a type, making an optional field required, tightening validation, or changing what a value means while keeping its name. The last one is the most dangerous, because nothing about the shape signals it.

When a breaking change is unavoidable:

Approach	Example	Notes
Path	<code>/v1/orders</code>	Most common, visible, easy to route and cache
Header	<code>Accept: application/vnd.example.v2+json</code>	Cleaner URLs, harder to test by hand
Query parameter	<code>/orders?version=2</code>	Simple, easily lost when URLs are copied

Path versioning is the usual choice for public APIs because it is obvious in a log line and easy to route at a gateway. Whichever is used, the migration matters more than the mechanism: publish the new version, run both, tell callers, measure who is still on the old one, and only then remove it. That expand-then-contract sequence is the same shape as a safe database migration.

10. REST, gRPC and GraphQL

	REST over JSON	gRPC	GraphQL
Transport	HTTP/1.1 or HTTP/2	HTTP/2 only	Usually HTTP, one endpoint
Payload	JSON text	Protocol Buffers, binary	JSON text

	REST over JSON	gRPC	GraphQL
Contract	OpenAPI, often written after	<code>.proto</code> file, contract first	Schema, contract first
Browser support	Direct	Needs a proxy such as gRPC-Web	Direct
Streaming	Limited	Built in, both directions	Subscriptions
Caching	HTTP caching works	Not through HTTP caching	Harder, since one endpoint and usually POST
Suits	Public APIs, wide clients	Service to service, low latency	Varied clients needing different fields

gRPC generates client and server code from a `.proto` definition, so the contract is compiled rather than described. The binary format and HTTP/2 multiplexing make it efficient between services, and the generated stubs remove a class of hand-written mapping bugs. The costs are that browsers cannot call it without a proxy, a binary body is harder to inspect during an incident, and every consumer needs the generated code.

GraphQL lets the client ask for exactly the fields it wants from one endpoint, which removes both over-fetching and the round trips caused by under-fetching. The costs are real: traditional HTTP caching gets harder, because many different queries share one endpoint and are commonly sent as `POST`, so the usual model of caching by URL and method does not apply. Resolvers make the N+1 problem easy to reintroduce, so a batching loader becomes necessary, and a deeply nested query can be expensive enough to need depth and complexity limits.

The answer that lands in an interview is about clients and coupling: REST for public and varied consumers, gRPC between services we control, GraphQL when many different clients need different shapes of the same data.

They are also not exclusive, and saying so is worth a mark. A common arrangement is REST at the public edge, gRPC between internal services, and a GraphQL layer in front for client applications that need to aggregate across several of them.

11. Authentication and authorization

Authentication establishes who the caller is. **Authorization** decides what that caller may do. They map onto 401 and 403 respectively, and mixing them up is a common interview slip.

Two ways to carry identity:

Session cookies. The server creates a session, stores it, and gives the client an identifier in a cookie. State lives on the server, so revoking is immediate. It needs shared session storage once there is more than one instance, and the cookie needs `HttpOnly`, `Secure` and `SameSite` set.

Tokens, usually JWT, JSON Web Token. A token carries the claims itself, in three base64url segments: header, payload, signature. The server verifies the signature and validates the claims, so no lookup is needed.

Verifying the signature alone is not enough, and this is a good detail to volunteer. A correctly signed token can still be the wrong token: issued by a different provider, intended for a different service, expired, or not yet valid. The checks that belong alongside the signature are `exp` for expiry, `nbf` for not-before, `iss` for the expected issuer, and `aud` for the audience this service answers to.

The precision worth having ready: **a JWT is signed, not encrypted, by default.** Anyone holding it can read the payload, so it must not contain anything secret. That is JWS, JSON Web Signature; the encrypted variant is JWE and is much less common.

The hard part of tokens is revocation. Because the server keeps no state, a token stays valid until it expires, so a logout or a compromised account does not immediately stop it. The usual answer is a short-lived access token, minutes rather than days, paired with a longer-lived refresh token that can be revoked because it is checked against storage.

OAuth 2.0 is a framework for delegated authorization: it lets a user grant one application limited access to their data in another, without sharing a password. It is not an authentication protocol, which is what **OIDC**, **OpenID Connect** adds on top of it, supplying an ID token describing who the user is.

```
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/orders/{id}")
public void delete(@PathVariable long id) { orders.delete(id); }
```

Role checks like this cover coarse permissions. They do not cover whether *this* user owns *this* order, which is the subject of the next section.

12. API security

The OWASP API Security Top 10 is a useful framework for the risks specific to APIs, and it is worth knowing the shape of it rather than only the famous entries:

Risk	What to remember
Broken object level authorization	Check access to the specific object, not just that the caller is logged in
Broken authentication	Weak token handling, no expiry, credentials in the wrong place
Broken object property level authorization	Do not let callers write or read fields they should not control
Unrestricted resource consumption	Cap payload size, page size, uploads and expensive queries
Broken function level authorization	Protect privileged and administrative operations, not just data

Risk	What to remember
Unrestricted access to sensitive business flows	Protect high-value actions from automated abuse
Server side request forgery	Never fetch a URL supplied by a caller without an allow list
Security misconfiguration	CORS, error verbosity, debug endpoints, missing headers
Improper inventory management	Old versions and forgotten environments still serving traffic
Unsafe consumption of APIs	Treat responses from upstream services as untrusted input

Four of these are worth code rather than a table row.

Broken object level authorization, also called IDOR, Insecure Direct Object Reference, sits at the top of the list and is the one worth naming first:

```
// Authenticated, and completely broken. Any logged-in user reads any order.
@GetMapping("/orders/{id}")
public OrderResponse get(@PathVariable long id) {
    return OrderResponse.from(orders.findById(id).orElseThrow());
}

// The ownership check is the actual authorization.
@GetMapping("/orders/{id}")
public OrderResponse get(@PathVariable long id, @AuthenticationPrincipal User user) {
    Order order = orders.findByIdAndCustomerId(id, user.getCustomerId())
        .orElseThrow(OrderNotFoundException::new);
    return OrderResponse.from(order);
}
```

Being logged in is not permission to read a specific row. Returning 404 rather than 403 for someone else's resource also avoids confirming that it exists.

Mass assignment. Binding a request body straight onto an entity lets a caller set fields nobody intended to expose:

```
// A caller sends {"email": "...", "role": "ADMIN"} and becomes an admin.
@PostMapping("/users")
public User create(@RequestBody User user) { return users.save(user); }
```

A request DTO, Data Transfer Object, containing only the fields a caller may set is the fix, and it is the same reason entities should not be returned either.

Broken object property level authorization, which older OWASP lists called excessive data exposure.

Serialising an entity directly sends every field, including the password hash, internal flags and anything a later migration adds. A response DTO makes the output an explicit decision, and the same category covers the write

direction, where a caller sets a property they should not control.

Injection. Parameterized queries and bound parameters, never string concatenation. The same applies to sort and filter parameters that reach a query builder.

Server side request forgery, SSRF. An endpoint that fetches a URL the caller supplied can be pointed at internal addresses the caller could never reach directly, including cloud metadata endpoints. Anything taking a URL needs an allow list of hosts and schemes rather than a block list.

Security misconfiguration. A permissive CORS, Cross-Origin Resource Sharing, policy that reflects any origin, verbose error responses in production, and debug endpoints left exposed.

Section 13 covers unrestricted resource consumption, since rate limiting is most of the answer to it.

13. Rate limiting

Rate limiting protects a service from being overwhelmed, shares capacity fairly between callers, and caps the cost of abuse. Four algorithms are common:

Algorithm	How it works	Trade
Fixed window	Count per clock interval, reset at the boundary	Simple, allows a double burst across a boundary
Sliding window	Count over a rolling interval	Smoother, more state to keep
Token bucket	Tokens refill steadily, each request spends one	Allows controlled bursts, common default
Leaky bucket	Requests drain at a fixed rate	Smooths output, queues or drops the excess

The response is `429 Too Many Requests`, and it should carry `Retry-After` so a well-behaved client knows when to return rather than guessing.

Two practical points. Broad protection belongs at the edge, in a gateway or proxy, before traffic reaches application threads, and business-specific limits belong in the application, because only it knows what the operation costs. A gateway caps a client at a hundred requests a second; the service caps password resets at five an hour and expensive report generation at ten a minute. And in a multi-instance deployment the counter has to be shared, usually in Redis, since per-instance counters multiply the real limit by the number of instances.

14. Timeouts and retries

Every remote call needs a timeout. Several Java clients default to waiting indefinitely, and an indefinite wait is how one slow dependency exhausts a thread pool and takes down a service that was otherwise healthy.

```
@Bean
RestClient inventoryClient(RestClient.Builder builder) {
    var factory = new SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(Duration.ofSeconds(2));
    factory.setReadTimeout(Duration.ofSeconds(3));
    return builder.requestFactory(factory).baseUrl("https://inventory").build();
}
```

Retries need three conditions before they help rather than hurt:

- **Only transient failures.** A timeout, a 503, a connection reset. Retrying a 400 will fail the same way every time.
- **Only idempotent operations**, or a `POST` protected by an idempotency key.
- **Exponential backoff with jitter.** Backoff spaces attempts out; jitter, meaning a random offset, stops every client that failed at the same moment retrying in step and producing a second outage.

A **circuit breaker** stops calling a dependency that is clearly failing, so requests fail immediately rather than each burning a timeout. It sits closed while things are healthy, opens once a failure threshold is crossed, and after a delay allows a trial request through to decide whether to close again. Resilience4j provides this in Spring, along with a **bulkhead**, which caps how many concurrent calls one dependency may consume so it cannot starve the rest of the service.

15. Idempotency keys

`POST` is not idempotent, so a client that times out cannot safely retry a payment or an order. The standard answer is to let the client supply a key that makes the operation idempotent:

```
POST /payments
Idempotency-Key: 8f14e45f-ea1d-4b1a-9a2c-0e7e02e5e1a3
```

The server stores the key with the result of the first successful request. A repeat with the same key returns that stored result rather than performing the work again.

The obvious implementation has a race in it, and it is worth being able to describe:

```
// BROKEN. Two concurrent retries both find nothing and both charge the card.
if (idempotency.find(key).isEmpty()) {
    PaymentResponse response = payments.charge(request);
    idempotency.save(key, response);           // one of the two saves now fails
    return ResponseEntity.ok(response);
}
```

A unique constraint on the key stops the second **record** from being written. It does nothing about the second **charge**, which already happened. The constraint fires after the side effect, which is too late.

The fix is to **claim the key atomically before performing the side effect**, so only one request ever reaches it:

```
@PostMapping("/payments")
public ResponseEntity<PaymentResponse> pay(
    @RequestHeader("Idempotency-Key") UUID key,
    @Valid @RequestBody PaymentRequest request) {

    // 1. Claim first. The INSERT either succeeds, making this request the
    //    owner, or the unique constraint rejects it and someone else owns it.
    if (!idempotency.claim(key)) {
        return idempotency.find(key)
            .filter(IdempotencyRecord::isComplete)
            .map(record -> ResponseEntity.ok(record.response()))
            // Claimed but unfinished: the first request is still running.
            .orElseGet(() -> ResponseEntity.status(HttpStatus.CONFLICT).build());
    }

    // 2. Only the owner gets here. Pass the same key downstream, so the
    //    provider can reject a duplicate too.
    PaymentResponse response = payments.charge(request, key);

    // 3. Record the outcome against the key already claimed.
    idempotency.complete(key, response);
    return ResponseEntity.ok(response);
}
```

Four details separate a correct implementation from an approximate one:

- **Claim before the side effect**, using an atomic operation such as an insert against a unique constraint. This is the whole point, and the ordering is what makes it work.
- **Handle the in-progress case**. A key claimed but not yet completed means the first request is still running. Returning 409 is one valid design, and so is waiting for the original to finish, or answering 202 with a status the caller can poll. What matters is that the contract picks one, rather than falling through and charging again.
- **Recover from a crash between the side effect and the completion record**, which otherwise leaves a key claimed forever. The claim needs recovery semantics, usually an expiry, but expiry alone is not sufficient: once it lapses we still do not know whether the charge went through. Retrying is only safe because the same key went to the provider, so their own idempotency rejects the duplicate, or because a reconciliation step asks them what actually happened.
- **Scope keys to the caller and expire them**. One client should not be able to collide with or replay another's, and keys kept forever are a slow leak.

16. Caching

HTTP caching lets clients and proxies avoid asking again:

```
Cache-Control: public, max-age=300  
ETag: "a3f5c9"
```

An `ETag` is a version tag for a representation. The client sends it back as `If-None-Match`, and if nothing changed the server answers `304 Not Modified` with no body, which saves the transfer but not the lookup. `Cache-Control: no-store` is the one to use for anything sensitive, since `no-cache` means revalidate rather than do not keep.

Server-side caching sits in front of expensive work:

```
@Cacheable(value = "orders", key = "#id")  
public Order findById(long id) { return repository.findById(id).orElseThrow(); }
```

The choices worth being able to discuss:

- **Local against distributed.** An in-process cache is fastest and gives every instance a different view. A shared cache such as Redis is consistent across instances and adds a network hop.
- **Invalidation strategy.** Cache-aside is the common pattern: read the cache, fall back to the source, then populate. Write-through updates both on write. A time to live, TTL, bounds how stale data can get without any invalidation logic at all, and is often enough.
- **Stampede.** When a popular key expires, every request misses at once and hits the source together. Staggered expiry, or a lock so one caller repopulates while the others wait, prevents it.

The honest summary for an interview: caching trades freshness and extra complexity for lower latency and less load on the source. It does not make data wrong, since a suitable time to live or invalidation strategy bounds how stale it can get. The question is always how stale the data is allowed to be, and anything that tolerates none should not be cached.

17. Layering a Spring Boot service

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService orders;

    public OrderController(OrderService orders) {        // constructor injection
        this.orders = orders;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public OrderResponse create(@Valid @RequestBody CreateOrderRequest request) {
        return OrderResponse.from(orders.create(request.toCommand()));
    }
}
```

Three layers, each with one job. The **controller** handles transport: parsing, validation, status codes. The **service** holds the business rules and the transaction boundary, and knows nothing about HTTP. The **repository** handles storage.

Two rules that carry most of the value:

DTOs at the edge, entities inside. A request DTO limits what a caller can set, and a response DTO makes the exposed fields an explicit decision. Returning entities from a controller couples the API contract to the database schema, so a column rename becomes a breaking API change.

Constructor injection, not field injection. The dependencies are visible in the signature, the fields can be `final`, and the class can be constructed in a test without a Spring context.

18. Error handling

Domain code throws meaningful exceptions. One place maps them onto HTTP.


```

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail handleNotFound(OrderNotFoundException e) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, e.getMessage());
        problem.setTitle("Order not found");
        problem.setType(URI.create("https://api.example.com/problems/order-not-found"));
        return problem;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail handleInvalid(MethodArgumentNotValidException e) {
        ProblemDetail problem = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        problem.setTitle("Validation failed");
        problem.setProperty("errors", e.getBindingResult().getFieldErrors().stream()
            .collect(toMap(FieldError::getField,
                FieldError::getDefaultMessage)));
        return problem;
    }
}

```

Keeping the mapping in one class means every endpoint returns the same error shape, and controllers stay free of `try` blocks. The catch-all handler for unexpected exceptions should log the detail and return a generic 500, so the internals stay internal.

19. Testing an API

Level	Tool	Covers
Unit	Plain JUnit, hand-written fakes	Business rules in the service, no Spring
Slice	<code>@WebMvcTest</code> with <code>MockMvc</code>	Routing, serialisation, validation, status codes
Integration	<code>@SpringBootTest</code> with Testcontainers	The whole path including a real database
Contract	Spring Cloud Contract, Pact	That a consumer and provider still agree
End to end	A deployed environment, driven over HTTP	One critical user flow across the real system

```

@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean OrderService orders;

    @Test
    void rejectsNegativeTotal() throws Exception {
        mvc.perform(post("/orders")
            .contentType(APPLICATION_JSON)
            .content("""
                {"customerId": 1, "total": -5}
                """))
            .andExpect(status().isBadRequest());
    }
}

```

Most tests belong at the bottom two levels, because they are fast and precise. Integration tests against a real database in a container are worth having for the paths that matter, and Testcontainers makes them reproducible rather than depending on whatever is installed locally. End-to-end tests are the slowest and most brittle, so they are worth keeping to a handful covering the flows that would be worst to break.

20. Observability

Three signals answer different questions, and being able to separate them is a good sign in an interview.

Logs say what happened in one request. Make them structured rather than sentences, so they can be queried, and put a correlation identifier on every line so one request can be followed across services.

Metrics say how the system behaves in aggregate. The **RED** set covers most needs: **R**ate of requests, **E**rrors, and **D**uration. Micrometer is the usual Java facade, exporting to Prometheus or similar.

Traces say where the time went across services. Each request carries a trace identifier through every hop, so a slow response can be attributed to the exact dependency responsible. The two identifiers are worth separating: a **trace identifier** names the whole distributed request, and a **span identifier** names one operation inside it, so the trace is the tree and the spans are its nodes. OpenTelemetry is the standard, and Micrometer Tracing wires it into Spring.

Health endpoints deserve one distinction. **Liveness** answers whether the process should be restarted.

Readiness answers whether it should receive traffic. Conflating them causes restarts when a dependency is briefly unavailable, when the correct response was to stop routing traffic and wait.

21. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What makes an API RESTful?"

REST is a set of constraints rather than a specification, and the ones that matter in practice are that the client and server are separate, that it is stateless, that responses say whether they are cacheable, and that there is a uniform interface where resources are identified by URL and manipulated with the standard methods. Statelessness is the one with the real consequence, because when no request depends on session state held in one instance's memory, any instance can serve any request, which is what makes horizontal scaling and rolling deploys straightforward. I would also be honest that most APIs called REST, including ones I have built, are really HTTP JSON APIs sitting at level two of the Richardson maturity model, using resources and methods properly but not hypermedia, because level three rarely pays for itself.

"What is the difference between safe and idempotent?"

Safe means the request does not change server state, so `GET`, `HEAD` and `OPTIONS`. Idempotent means repeating the request leaves the server in the same state as doing it once, so `GET`, `PUT`, `DELETE`, `HEAD` and `OPTIONS` are, `POST` is not, and `PATCH` depends on how it is written, since setting a field is idempotent and incrementing one is not. Every safe method is idempotent, and the reverse is not true, because `DELETE` changes state but repeating it changes nothing further. One detail worth adding is that `DELETE` is still idempotent even though the second call returns 404 rather than 204, because idempotency is about the resulting state and not about identical responses. The reason it all matters is retries, since a client whose request timed out does not know whether it was processed.

"When would you return 401 and when 403?"

401 says the caller has not been identified, so either no credentials arrived or they were not valid. 403 says the caller has been identified and is refused anyway. The naming is confusing because 401 reads Unauthorized when it actually means unauthenticated. One thing I would add is that for a resource belonging to another user, returning 404 rather than 403 is often better, because a 403 confirms that the resource exists, which leaks information to someone probing identifiers.

"How would you paginate a large collection?"

Offset pagination with page and size is the familiar option, and it is fine for an admin table where jumping to a specific page matters. It has two costs: the database walks and discards every skipped row so deep pages get slower, and if rows are inserted between requests the pages shift so an item can be seen twice or missed. Keyset or cursor pagination uses the last item seen as the starting point, so the cost is flat however deep it goes and the results stay stable while data changes, at the price of not being able to jump to page fifty. I would choose keyset for feeds and anything large, and offset where random access to pages is a requirement. Either way I would cap the page size on the server, and I would design for pagination in the first version, because retrofitting it usually breaks clients: a bare JSON array becomes an object with the items and a cursor inside it, and that changes the type every client parses.

"How do you make a POST safe to retry?"

With an idempotency key. The client generates a unique key and sends it as a header, and the server stores that key alongside the result of the first successful request, so a retry with the same key returns the stored result instead of doing the work twice. That is what stops a timed-out payment from being charged twice when the client retries. The detail that makes it actually correct is the ordering: the key has to be claimed atomically before the side effect, not saved after it. If the code checks whether the key exists, charges the card and then writes the record, two concurrent retries both find nothing and both charge, and the unique constraint only rejects the second write once the money has already moved. So the first step is an insert that either wins or loses, and only the winner performs the charge. After that it is about the edges: a key claimed but not completed means the first request is still running, so 409 rather than charging again, and a crash between charging and recording needs the claim to expire, which is safe if the same key was passed to the payment provider.

"A downstream service is slow. What happens to your service, and what do you do?"

If the call has no timeout, the request thread waits, and under load every thread ends up waiting on that dependency, so a service that is otherwise healthy stops serving anything. The first fix is a timeout on every remote call, because several Java clients wait indefinitely by default. Then retries, but only for transient failures and only for idempotent operations, with exponential backoff and jitter, since without jitter every client that failed at the same moment retries in step and causes a second outage. Beyond that a circuit breaker stops calling a dependency that is clearly failing so requests fail immediately instead of each burning a timeout, and a bulkhead caps how many concurrent calls that one dependency can consume so it cannot starve the rest of the service.

"REST or gRPC?"

It depends on who the caller is. REST over JSON for public APIs and varied clients, because anything can call it, it is readable during an incident, and HTTP caching works. gRPC between services we control, because the contract is a `.proto` file that generates both sides so the mapping code is not hand-written, the binary format over HTTP/2 is efficient, and streaming is built in rather than bolted on. The costs of gRPC are that browsers cannot call it without a proxy such as gRPC-Web, and that a binary body is harder to inspect when something is wrong at two in the morning. GraphQL is the third option, and it fits when many different clients need different subsets of the same data, at the cost of traditional HTTP caching being harder, since many queries share one endpoint and usually arrive as `POST`, and resolvers making N+1 easy to reintroduce.

"What is the most common API security mistake you look for?"

Broken object level authorization, sometimes called IDOR. It is when an endpoint checks that the caller is authenticated but not that the caller owns the specific resource, so any logged-in user can read any order by changing the identifier in the URL. It sits at the top of the OWASP API Security Top 10 and it is easy to write by accident, because the endpoint looks correct and the tests pass when the test user happens to own the data. The fix is to scope the query by the authenticated principal rather than filtering afterwards, so the lookup is by order id and customer id together, and to return 404 rather than 403 so the response does not confirm that someone else's resource exists. The related one I check for is mass assignment, where binding a request body straight onto an entity lets a caller set a field like role.

"How would you version an API?"

By avoiding it for as long as possible, because the cheapest version is the one never needed. Adding an endpoint or an optional field is not a breaking change as long as clients ignore unknown fields, so I would insist on tolerant readers early. What does break callers is removing or renaming a field, changing a type, making an optional field required, or changing what a value means while keeping its name, and that last one is the most dangerous because nothing about the shape signals it. When a break is unavoidable I would use a version in the path, because it is obvious in a log line and easy to route at a gateway, and then treat it as a migration rather than a release: publish the new version, run both, measure who is still on the old one, and only then remove it.

22. Common mistakes

- Returning 200 with an error in the body. Every proxy, monitor and retry policy reads the status code, so a success code on a failure defeats all of them.
- Confusing 401 and 403, or returning 403 for another user's resource and confirming that it exists.
- Binding a request body directly onto an entity, which lets a caller set fields such as a role, and returning entities from controllers, which exposes every column and couples the contract to the schema.
- Checking authentication but not ownership, so any logged-in caller can read any record by changing an identifier.

- Shipping a collection endpoint with no pagination, then finding that adding it is a breaking change.
- Making a remote call with no timeout, which is how one slow dependency exhausts a thread pool.
- Retrying without backoff and jitter, so every client that failed together retries together.
- Retrying a non-idempotent `POST` without an idempotency key, and charging a card twice.
- Implementing an idempotency key by checking, performing the side effect, then saving. The key must be claimed before the side effect, or two concurrent retries both charge and the unique constraint only rejects the second record.
- Putting anything secret in a JWT payload, which is signed rather than encrypted and readable by anyone holding it.
- Using `no-cache` when the intent was `no-store`. The first means revalidate, the second means do not keep it at all.
- Rate limiting per instance rather than through shared state, which multiplies the real limit by the number of instances.
- Using one endpoint for both liveness and readiness, so a brief dependency failure restarts a healthy process instead of pausing traffic to it.

23. Quick reference

Term	Meaning
Safe	Does not change server state. <code>GET</code> , <code>HEAD</code> , <code>OPTIONS</code>
Idempotent	Repeating it leaves the same state. Adds <code>PUT</code> , <code>DELETE</code>
401 against 403	Not authenticated, against authenticated but not permitted
400 against 422	Cannot be parsed, against parsed but breaks a rule
201 Created	Include a <code>Location</code> header pointing at the new resource
429	Rate limited. Include <code>Retry-After</code>
Statelessness	No session held on the server, so any instance serves any request
Richardson level 2	Resources plus proper methods and status codes, without hypermedia
Offset pagination	<code>page</code> and <code>size</code> . Random access, deep pages slow, results shift
Keyset pagination	Cursor from the last item. Flat cost, stable, no page jumping
RFC 9457	Problem Details, the standard error body shape
Tolerant reader	A client that ignores unknown fields, so additions do not break it
JWT	Signed, not encrypted. Readable by the holder, hard to revoke
OAuth 2.0 against OIDC	Delegated authorization, against authentication on top of it
IDOR	Authenticated but unauthorized access to another user's object

Term	Meaning
Mass assignment	Binding a request body onto an entity, letting callers set hidden fields
Token bucket	Steady refill, spend one per request. Allows controlled bursts
Jitter	Randomised backoff, so retries do not synchronise
Circuit breaker	Stop calling a failing dependency, fail fast, probe to recover
Bulkhead	Cap concurrent calls to one dependency so it cannot starve the rest
ETag	Version tag. Returned as <code>If-None-Match</code> , answered with 304
Cache-aside	Read cache, fall back to source, then populate
RED	Rate, errors, duration. A useful baseline for request-driven services
Liveness against readiness	Restart the process, against stop sending it traffic

Symptom	Likely cause
A retry creates a duplicate order or payment	A non-idempotent <code>POST</code> with no idempotency key, or a key claimed after the side effect rather than before
Thread pool exhausted, service unresponsive	A remote call with no timeout
A dependency recovers, then immediately falls over	Retries without jitter, all clients returning at once
A caller sets a field the API never documented	Mass assignment from binding onto an entity
One user can read another's records	Ownership never checked, only authentication
Deep pages get slower and slower	Offset pagination walking discarded rows
Items appear twice across pages	Offset pagination while rows are being inserted
Rate limit allows several times the configured rate	Per-instance counters instead of shared state
Clients break when a field is added	Clients are not tolerant readers
A healthy pod restarts during a brief outage	Liveness and readiness served by one endpoint